

DOCUMENTAÇÃO TÉCNICA

DispatchBot

Arquitetura, funcionamento interno, contratos de dados, configuração, build e operação do robô que abre chamados de GMUD e Backup no ServiceNow da Cirion.

Versão do código	**1.10.6**
Público	**Time de RPA / desenvolvimento e sustentação**
Repositório	**github.com/viniciusisliker/DispatchBot**
Pilha	**Python 3.12 · Playwright · pystray · CustomTkinter · PyInstaller · Inno Setup 6**
Integrações	**Power Automate · SharePoint (API REST) · ServiceNow Employee Center · Teams**

Sumário

1 Visão geral e arquitetura

2 Ciclo de vida de um alerta

3 Processo, threads e sincronização

4 Loop principal

5 Processamento de um alerta

6 Preenchimento e envio no ServiceNow

7 Idempotência: nunca dois chamados

8 Fila no SharePoint (API REST)

9 Navegador, sessão e login SSO

10 Plantão entre máquinas e histórico do time

11 Parada de emergência

12 Interface: bandeja, painel e janelinha

13 Configuração (config.json)

14 Contratos de dados (JSON)

15 Fluxos do Power Automate

16 Tratamento de erros

17 Arquivos locais, logs e diagnóstico

18 Build, instalador e Releases

19 Ferramentas: Mapear, --testar e Demo

20 Testes e manutenção

21 Decisões fechadas, limites e pendências

Como ler este documento

As seções 1 a 5 explicam o caminho de um alerta de ponta a ponta. As seções 6 a 12 detalham cada módulo. As seções 13 a 15 são referência (config, formatos dos arquivos e fluxos). As seções 16 a 21 cobrem operação, build e manutenção. Nomes entre `crases` são nomes reais do código (`arquivo.py`, `função`, `chave do config`).

Documentos relacionados

- **Manual do plantonista** (`Manual-DispatchBot.pdf`, também anexo às Releases): uso do dia a dia, sem termos técnicos.
- **README.md**: instalação, mapeamento de campos, testes rápidos.
- **docs/power-automate.md**: passo a passo dos fluxos de nuvem (expressões completas).
- **fluxos/**: pacotes exportados dos três fluxos (.zip) e como importar.

1 Visão geral e arquitetura

O DispatchBot é um robô RPA de desktop (Windows) que transforma alertas de GMUD e de falha de Backup em chamados no ServiceNow da Cirion, pelo Employee Center (`/esc`). Não existe API do ServiceNow disponível para o time: o robô usa um navegador real (Edge ou Chrome do sistema) com a sessão SSO do plantonista.

OUTLOOK	POWER AUTOMATE	SHAREPOINT	MÁQUINA DO N1	POWER AUTOMATE → TEAMS
E-mail do alerta	**Fluxo de entrada**	**Fila**	**DispatchBot**	**Retorno**
GMUD (ServiceNow YDUQS) ou falha de Backup (CSV do NetBackup).	"GMUD | Todos" ou BackupBot: posta o card no Teams e grava um JSON em `pendentes`.	`DispatchBot-Fila`: pastas `pendentes` → `processando` → `processados` | `erro` + `retorno`.	Lê a fila pela API REST, preenche o formulário no navegador, envia e captura o número.	"DispatchBot | Retorno" lê `retorno` e posta ✅ número + link (ou ⚠️ motivo) no chat do time.

Componentes

COMPONENTE	ONDE RODA	RESPONSABILIDADE
GMUD | Todos	Power Automate (nuvem)	Gatilho na caixa da operação (e-mail "Uma tarefa de Mudança foi atribuída"). Extrai CHG, data, descrição e o time (`SN_TI_CHG_EXECUTOR_CIRION_<TIME>` ou `..._Backup_Cirion`), posta o card no chat do time e grava `<TIME>_*.json` em `pendentes`.
BackupBot	Power Automate (nuvem)	Gatilho na caixa do Backup. Lê o CSV anexo, filtra as falhas, posta o card e grava um `BACKUP_*.json` por linha de falha.
DispatchBot (robô)	Máquina do Dispatcher N1 (Windows, por usuário)	Fila, reserva, idempotência, preenchimento, envio, captura do número, retorno, painel, histórico do time, parada de emergência.
DispatchBot | Retorno	Power Automate (nuvem)	Gatilho em arquivo novo em `retorno`; Switch pelo `tipo` posta a mensagem no chat em grupo certo.
DispatchBot-Mapear	Máquina do desenvolvedor/N1	Ferramenta de console: lista campos de um formulário, testa o preenchimento sem enviar (`--testar`).

Princípios de projeto

- **O robô não inventa dados.** Faltou CHG, data ou descrição, ou a CHG não segue `CHG\d{7}`: não abre, vai para `/erro` e o Teams avisa.
- **Nunca dois chamados para o mesmo alerta.** Histórico local com `fsync` antes do clique em Enviar; status `enviando` bloqueia qualquer reenvio automático.
- **Nunca digita senha.** No login, só clica em "Use external login" e em "Submit"; o resto (conta Microsoft, Windows Hello) é do plantonista.
- **Um robô ativo por vez.** Só o Dispatcher N1 da vez fica ativo; os outros ficam em Pausado. A reserva atômica no SharePoint protege mesmo se dois ficarem ativos.

- **Formulários descritos no config.** Cada tipo de alerta é um bloco de `tipos` (URL, passos, seletores, captura do número). Tipo novo não exige código.
- **Falhar para o lado seguro.** Na dúvida (obrigatório vazio, conferência falhou), o robô não envia e chama o plantonista.

Mapa do código (`robo/`)

ARQUIVO	LINHAS	PAPEL
`main.py`	~1300	Classe `Robo`: loop, ciclo, `processar`, guardas, novas tentativas, plantão, histórico do time, emergência, supervisor; `main()` com mutex e modo `--vigia`.
`servicenow.py`	~385	Placeholders, validação de dados, execução dos passos, detector de obrigatórios (`JS_OBRIGATORIOS`), envio/conferência, captura do número, link de tratamento.
`fila_sharepoint.py`	~390	`FilaSharePoint`: fila pela API REST (MoveTo atômico, digest, renovação de sessão), status, plantão, histórico.
`fila.py`	~305	`Fila` em pasta (modo antigo, OneDrive), `Historico` (`processados.jsonl`), constantes `HOST` / `SEP`.
`navegador.py`	~340	`Navegador`: Edge→Chrome com perfil próprio, detecção e espera de login, cookies do SharePoint, limpeza de processos órfãos.
`painel.py`	~800	Painel CustomTkinter (thread própria), janelinha "Precisa de você", janela do nome, tela Histórico do time, `painel.jsonl`.
`bandeja.py`	~140	Ícone pystray (avatar + bolinha de status) e menu.
`config.py`	~140	Carregar/mesclar/salvar config, herança de tipos, nome do plantonista.
`emergencia.py`	~80	Atalho global (RegisterHotKey) e arquivo PARAR na pasta local.
`log.py`	~55	Log rotativo e screenshots de erro.
`pasta.py`	~155	Só no modo antigo: localizar/sincronizar a pasta do OneDrive.
`mapear_campos.py`	~175	`DispatchBot-Mapear.exe`: mapear campos e `--testar`.
`demo.py`	~100	Demonstração: preenche GMUD e Backup no portal real sem enviar.
`versao.py`	2	Fonte única da versão (`VERSAO = "1.10.6"`).

2 Ciclo de vida de um alerta

Um alerta é um arquivo JSON. O estado dele é a pasta onde está e, para a idempotência, a última linha do seu `id` no histórico local da máquina.

`pendentes/INFRA_x.json` → **reservar (MoveTo)** → `processando/INFRA_x__HOST.json` → `processados/RITM0421153_INFRA_x.json` **ou** `erro/INFRA_x.json` + `retorno/INFRA_x.json`

PASTA	QUEM GRAVA	CONTEÚDO
`pendentes`	Fluxos de entrada	Alerta original. O robô ignora arquivos com menos de `idade_minima_arquivo_seg` (5 s) e nomes iniciados por `~` ou `.`. Ordem: mais antigo primeiro.
`processando`	Robô (reserva)	`<nome>__<HOST>.json`. A data do arquivo é regravada na reserva: é ela que as outras máquinas usam para detectar reserva abandonada.
`processados`	Robô	Alerta + `resultado` {numero, url, maquina, processado_em}. Nome `<NUMERO>_<nome>.json`; sem número (ignorado, aberto à mão) fica só `<nome>.json`.
`erro`	Robô	Alerta + `erro` {mensagem, maquina, em}. JSON ilegível: o arquivo original + `<nome>.erro.txt` com o motivo.
`retorno`	Robô	Resultado para o Teams (status `ok` ou `erro`), lido pelo fluxo Retorno.
`status`	Cada robô	`<HOST>.json`: batimento a cada 5 min ou na troca de status.
`plantao`	Robô que assume	`<HOST alvo>.json`: pedido para o robô alvo pausar.
`historico`	Cada robô	`<AAAA-MM-DD>_<HOST>.jsonl`: eventos do dia da máquina.
`PARAR*` (raiz)	Pessoa	Parada remota de todos os robôs; a 1ª linha vira o motivo.

As subpastas que faltarem são criadas pelo robô (`verificar_pasta`: a cada 5 min no SharePoint, a cada ciclo na pasta local). A raiz `DispatchBot-Fila` nunca é criada pelo robô: se não existir, é erro de configuração.

Desfechos possíveis

DESFECHO	DESTINO	RETORNO (TEAMS)	HISTÓRICO LOCAL
Chamado aberto	`processados/<NUM>_...`	✅ número + link de tratamento	`ok` com numero/url
Arquivo repetido (id já `ok`)	`processados`	nenhum	sem mudança
Cliente em `ignorar_clientes`	`processados`	nenhum	`ok` + `ignorado`

Alerta antigo: "já foi aberto à mão"	`processados`	nenhum	`ok` + `manual`
Dado faltando, tipo não mapeado, texto ilegível, recusa do formulário	`erro`	⚠️ motivo	`erro`
Tipo desconhecido	`erro`	⚠️ motivo	sem mudança (`erro_final` com id vazio)
Falha depois do clique em Enviar / ninguém conferiu	`erro`	⚠️ "confira no ServiceNow"	mantém `enviando`
Pulado pelo plantonista	`erro`	⚠️ "o plantonista pulou este alerta...; o chamado não foi enviado. Abra à mão."	mantém `enviando`
Login pendente, pausa, emergência antes do envio	volta a `pendentes`	nenhum	sem mudança
JSON inválido	`erro` + `.erro.txt`	nenhum (sem dados para o Teams)	sem mudança

3 Processo, threads e sincronização

Um processo por usuário (`DispatchBot.exe`, sem console). O ícone da bandeja ocupa a thread principal; todo o Playwright roda numa única thread ("robo"). As demais threads só trocam sinais com ela.

THREAD	ORIGEM	FAZ
principal	`Bandeja.rodar` (pystray)	Loop de mensagens do ícone e do menu. Callbacks do menu rodam aqui e só mexem em config/flags.
robo	`Robo.loop`	Único dono do Playwright e da fila (incluindo renovar a sessão do SharePoint, que abre o navegador).
supervisor	`supervisionar`	A cada 30 s: se a thread "robo" morreu, descarta o navegador (objetos do Playwright pertencem à thread antiga) e cria outra.
evento-sair	`aguardar_evento_sair`	Espera o evento nomeado `DispatchBot_Sair` (sinalizado pelo instalador) e encerra com segurança.
parada-remota	`vigiar_parada_remota`	Procura o arquivo `PARAR` a cada 10 s (SharePoint) ou 3 s (pasta). Usa a sessão já carregada, nunca renova (`pode_renovar=False`).
atalho	`emergencia.escutar_atalho`	`RegisterHotKey` + `GetMessageW`: dispara `parar_agora("atalho")`.
painel	`Painel._rodar`	Mainloop do Tk (CustomTkinter). Sobe escondida no início: a janelinha "Precisa de você" depende dela.
painel-inicio	`Robo.iniciar`	Curta: sobe o painel escondido sem atrasar o início do robô.
leitura do histórico	`Painel._carregar_historico`	Lê `historico/` do SharePoint para a tela Histórico do time sem travar o Tk (`pode_renovar=False`).
pergunta-alerta-antigo	`perguntar_por_caixa`	Só sem painel: `MessageBoxW` Sim/Não/Cancelar, uma por vez.

Como as threads conversam

- `threading.Event`: `parar` (encerrando), `acordar` (verificar agora, interrompe a espera do polling), `emergencia` (parada imediata, vista por todos).
- **Pedidos por atributo**, consumidos pela thread do robô: `pedido_conferencia` ("mostrar"/"pular"), `pedido_login_frente`, `pedido_assumir` (hosts), `respostas_antigos`. O painel só grava o pedido; quem toca no navegador é sempre a thread do robô.
- **Estado para o painel**: `estado_painel()` monta um dicionário (status, detalhe, pendentes, em andamento, decisões, conferência, login, nome, outros ativos) lido pelo painel a cada 1 s.
- **Config como fonte de verdade** de `pausado`, `emergencia` e `envio_automatico`: o menu grava o arquivo; o loop relê o config a cada ciclo (`config.carregar()`), então editar o `config.json` não exige reiniciar.
- Eventos do histórico do time ficam em `self.eventos` protegidos por `trava_eventos` (máx. 500 em memória) até a thread do robô gravar.

Início do processo (`main()`)

1. Mutex `DispatchBot_Mutex`: se já existe, sai (com `--vigia`, sai calado; sem, mostra "já está em execução").
2. Com `--vigia` e a flag `encerrado_pelo_usuario`: sai (o vigia respeita quem fechou pelo menu "Sair"). Sem `--vigia`, a flag é apagada.
3. Configura o log e o `threading.excepthook`; carrega o config (inválido = caixa com a linha do erro e sai).
4. Só no modo antigo (sem `fila_sharepoint.site`): acha/sincroniza a pasta do OneDrive.
5. `Robo().iniciar()` → ícone + threads. Ao sair: `logging.shutdown()` e `os._exit(0)` (a thread do Tk podia segurar o processo e travar o build/instalador).

4 Loop principal

`Robo.loop` repete a cada `intervalo_polling_seg` (15 s), ou antes quando o evento `acordar` é sinalizado (menu "Verificar agora", decisão do plantonista, troca de pausa).

```
# Robo.loop (resumo)
enquanto não encerrando:
    cfg = config.carregar()                        # relê o config a cada volta
    pergunta o nome na 1ª execução (painel)
    fila = abrir_fila(fila)                        # FilaSharePoint (padrão) ou Fila (pasta)
    fila.verificar_pasta()                         # a cada 5 min; cria subpastas
    tratar_plantao(fila)                           # grava pedidos "assumir"; lê pedido para esta
máquina
    gravar_eventos(fila)                           # histórico do time
    se pausado ou emergência:
        fecha o navegador; ícone cinza (ou vermelho na emergência); gravar_status
    senão:
        login_inicial() na 1ª vez, ou se o navegador "sempre aberto" estiver fechado
        ciclo(fila)
        manter_sessao()                            # recarrega o portal se parado há
manter_sessao_min
        fechar_se_ocioso()                         # só se fechar_apos_ocioso_min > 0
    espera acordar ou intervalo_polling_seg
```

`ciclo(fila)`

1. `recuperar_reservas(historico)`: arquivos desta máquina que ficaram em `processando` (robô fechado no meio). Se o histórico diz `ok` → conclui (com retorno só se houver número); `enviando` → `/erro` "confira no ServiceNow"; outro → devolve a `pendentes`.
2. `tratar_reservas_abandonadas(reserva_abandonada_min)`: reservas de **outras** máquinas paradas há mais de 60 min vão para `/erro` (não para `pendentes`: o chamado pode ter sido aberto antes da queda).
3. `gravar_status`: batimento, leitura dos outros robôs, aviso de "outro robô ativo" (no máx. 1 por hora por máquina).
4. Lista `pendentes`; para cada arquivo, nessa ordem: para se pausado/emergência ou se um colega assumiu o plantão (`verificar_pedido_plantao`, a cada 30 s, só entre alertas); pula alerta antigo sem decisão; chama `processar`. Se `processar` devolve `False` (ex.: login pendente), o ciclo para: os próximos alertas dependem da mesma coisa.

Exceções no loop

EXCEÇÃO	TRATAMENTO
`AguardandoPasta`	Só modo antigo: ícone azul "Preparando"; aviso único se a pasta não aparecer em 10 min.
`PastaIndisponivel`, `ConfigInvalido`	Ícone vermelho (`erro_infra`); some sozinho quando o problema passa.
Janela do navegador fechada (`"has been closed"`)	Não é erro: descarta o navegador, notifica e reabre no próximo ciclo.

Qualquer outra	`log.exception` + ícone vermelho com a mensagem amigável (`MENSAGENS_AMIGAVEIS`). Notificação repetida só se a mensagem mudar ou depois de 10 min.

No fim do loop (`finally`): fecha o navegador, grava o status `encerrado`, evento "encerrou" e descarrega os eventos pendentes.

5 Processamento de um alerta

`Robo.processar(fila, arquivo)` aplica as guardas em ordem fixa. A ordem importa: nada que possa abrir um chamado acontece antes das verificações de identidade, legibilidade e duplicidade.

#	ETAPA	REGRA	RESULTADO
1	Reservar	`fila.reservar` (MoveTo atômico). Falhou = outra máquina pegou.	pula o arquivo
2	Ler o JSON	Falha de conexão → devolve; conteúdo inválido → erro.	`/erro` + `.erro.txt`
3	Limpar texto	`limpar_texto`: remove caracteres de controle (menos `\n \r \t`) de todo o alerta. Motivo: CSV UTF-16 lido como UTF-8 no fluxo deixa `\0` entre as letras.	segue
4	Deduzir o tipo	Sem `tipo`: `grupo_executor` terminado em `BACKUP_CIRION` → `GMUD_BACKUP`; contendo `CIRION_` → o que vem depois (INFRA, DBA, SO...).	segue
5	Sem `id`	—	`/erro` ⚠️
6	Texto ilegível	`texto_ilegivel` em chg/titulo/descricao: caractere `�` ou > 20% de símbolos fora do comum. Título vira "(texto ilegível)", original em `titulo_original`.	`/erro` ⚠️ "abra à mão pelo e-mail original"
7	Tipo desconhecido	`config.tipo_efetivo` devolve None (tipos começando com `_` são modelos).	`/erro` ⚠️
8	Idempotência	Histórico `ok` → conclui sem retorno (arquivo repetido). `enviando` → erro "confira no ServiceNow". `forcar_reprocessamento: true` ignora as duas.	`processados` ou `/erro`
9	Cliente ignorado	`cliente_ignorado`: compara **só a coluna Cliente** (4ª coluna da linha do CSV, no `id` depois do `#`, ou 2º trecho do título `Política | Cliente | …`) com `ignorar_clientes` do tipo. As outras colunas (ex.: Master) se repetem em todo alerta e não entram (1.10.5). Compara o campo inteiro, sem maiúsculas; nome curto casa com FQDN.	`processados`, sem chamado e sem Teams
10	Alerta antigo	`recebido_em` há mais de `perguntar_alerta_antigo_min` (120): devolve o arquivo e pergunta (Abrir / Já foi aberto à mão / Decidir depois). "Depois" pergunta de novo em 30 min. Os alertas novos seguem.	espera / abre / `processados` (`manual`)
11	Login vencido	`login_pendente` e alerta com mais de `max_espera_login_min` (60).	`/erro` ⚠️ "abra manualmente"
12	Abrir o chamado	Até `tentativas_por_alerta` (3) vezes, com navegador novo e espera de `espera_entre_tentativas_seg × tentativa` (20 s, 40 s...). Detalhes na seção 6.	ver tabela abaixo

13	Concluir	Histórico `ok` → `painel.jsonl` → evento "chamado" → `fila.concluir` (processados + retorno). Se gravar na fila falhar, o chamado já está no histórico: o próximo ciclo refaz pela recuperação.	✅ no Teams

Exceções durante a abertura

EXCEÇÃO	SIGNIFICA	AÇÃO
`Interrompido`	Pausa ou emergência no meio	`interromper`: com `enviando` → `/erro`; senão devolve intacto.
`LoginNaoConcluido`	Ninguém fez login a tempo	Devolve a `pendentes`, ícone amarelo, interrompe o ciclo.
`TipoNaoMapeado`, `DadoFaltando`, `FormularioRecusado`	Repetir não adianta; nada foi aberto	Screenshot + `/erro` ⚠️.
`Pulado`	Plantonista clicou "Pular este alerta"	`/erro` ⚠️; mantém `enviando` (bloqueia reenvio).
`NaoConfirmado`	Conferência sem clique em `confirmacao_timeout_min` (30)	`/erro` ⚠️ "confira no ServiceNow"; mantém `enviando`.
Outra, com `enviando`	Falhou depois do clique	`/erro` ⚠️ "o chamado pode ter sido aberto". Nunca tenta de novo.
Outra, sem `enviando`	Falha passageira (rede, timeout)	Nova tentativa; na última, `/erro` ⚠️ "(3 tentativas)".

6 Preenchimento e envio no ServiceNow

`servicenow.abrir_chamado(nav, cfg, tipo_cfg, alerta, ...)` devolve `(numero, url)`. Tudo o que é específico de um formulário vem do bloco do tipo no config.

Variáveis e placeholders

Valores e buscas dos passos aceitam `{campo}` e `{campo|formato}` (regex `\{(\w+)(?:\|([^{}]+))?\}`). O dicionário de variáveis é montado por `variaveis()`, em prioridade crescente:

1. `padroes` do tipo (o alerta pode trocar; ex.: `servico_afetado`, `failure_type`, `impacto` do Backup);
2. campos simples do JSON do alerta (strings/números não vazios);
3. `id_curto` = 10 primeiros hex do SHA-1 do `id`;
4. `variaveis` do tipo (fixas: conta, caller, categoria, `time_titulo`). O alerta não sobrescreve.

Datas: `ler_data` aceita ISO (com `Z`) e `dd/MM/yyyy HH:mm[:ss]`, `dd/MM/yyyy`, `yyyy-MM-dd HH:mm[:ss]`. O formato de saída usa tokens do Excel (`yyyy dd MM HH mm ss`), ex.: `{data_execucao|dd/MM HH:mm}` → `29/04 09:30`.

Validações antes de abrir o navegador

- **Tipo não mapeado:** qualquer `PREENCHER` no bloco efetivo (sem as chaves `_ajuda`) → `TipoNaoMapeado`. Só no envio real; no `--testar` passa, para mapear aos poucos.
- **Dado faltando** (`faltando()`): todo placeholder usado em `url_formulario`, `valor` e `busca` (menos passos com `pular_se_vazio`) precisa existir e, se tiver formato de data, ser uma data válida. Depois, cada regex de `validar` precisa casar inteira (`re.fullmatch`), ex.: `chg: CHG\d{7}`. Qualquer problema → `DadoFaltando` listando todos.

Ações dos passos

AÇÃO	O QUE FAZ NO PLAYWRIGHT
`preencher`	`locator.fill(valor)`
`selecionar`	`select_option(valor)` (valor ou texto da opção) em `<select>` nativo
`referencia`	Campo select2 com busca: clica, digita `busca` (ou `valor`) com `press_sequentially` (60 ms por tecla, o select2 busca por tecla) em `.select2-drop-active input.select2-input` e clica na opção que **contém** `valor`.
`escolher`	Lista fixa select2: clica e escolhe a opção cujo texto é **exatamente** `valor` (Categoria `Other`, Failure Type).
`marcar`	`set_checked`; sem `valor` marca, com `false/0/não/vazio` desmarca.
`clicar`	`click()`
`aguardar`	Espera o seletor ficar visível ou `valor` ms.

Opções por passo: `mapa` (tradução de valor, `*` = padrão), `opcional` (falha vira aviso no log), `pular_se_vazio`, `seletor_busca` e `seletor_opcao` (sobrescrevem os do select2). Antes de cada passo,

`verificar_parada` levanta `Interrompido` se o robô foi pausado/parado. Falha num passo obrigatório vira `RuntimeError("Falha no passo N (...)")`.

Envio automático × conferência

O robô decide pelo grupo do tipo: `GMUD` para todo tipo que herda `_GMUD`, senão o próprio nome (`BACKUP`). Lê `envio_automatico[grupo]` (padrão `true`); o plantonista troca pelo menu do ícone. O resultado vai para `confirmar_antes_de_enviar` do tipo efetivo (a chave antiga no config não é mais lida).

1. Guarda os números que já estão na tela (`ja_na_tela`): não contam como o número novo.
2. **Envio automático:** só clica se `botao_enviar` estiver mapeado e `obrigatorios_vazios(page)` devolver lista vazia. O detector (`JS_OBRIGATORIOS`) procura `.form-group` / `.sp-form-field` visíveis com asterisco visível (`.mandatory`, `.fa-asterisk`), `is-required`, `[required]`, `aria-required` ou rótulo terminando em `*`; em campo de referência vale qualquer input não vazio do grupo. Obrigatório vazio, botão sem seletor ou erro na conferência = **não clica** e cai para a conferência com o motivo.
3. **Ponto sem volta:** `antes_de_enviar()` grava `enviando` no histórico (com `fsync`) **antes** do clique ou da conferência.
4. **Conferência:** traz o formulário para a frente, ícone amarelo, janelinha "Precisa de você" com os campos preenchidos (`resumo_campos`), o motivo e a contagem até `confirmacao_timeout_min`. Botões: "Mostrar formulário" (`bring_to_front`) e "Pular este alerta" (`Pulado`, só se nenhum número apareceu).
5. **Captura do número** (laço de 1 s até `timeout_seg` depois do clique do robô, padrão 60 s se faltar a chave; na conferência, até `confirmacao_timeout_min`): aplica `captura_numero.regex` ao texto de `seletor` (todos os elementos, mesmo escondidos) ou da página inteira; ignora os de `ja_na_tela` (números já presentes antes do envio, no texto visível **e** no do seletor); escolhe na ordem de `preferir`; sem o preferido, espera `espera_preferido_seg` (padrão 10 s se faltar a chave) e aceita o primeiro. No automático, se `seletor_erro_validacao` aparecer → `FormularioRecusado`.

TIPO	REGEX	PREFERIR	SELETOR	TEMPO
GMUD (Hosting)	`(RITM|CS)\d{7}`	RITM	`#req-item-toggle a, .ticket-id`	90 s; espera o RITM 30 s
BACKUP (Incidente)	`(INC|RITM|REQ|CS)\d{7}`	INC	`body` (página inteira, inclusive escondido)	120 s; espera o INC 60 s

O Hosting cria um caso **CS** e um **RITM** ligado (aba Related Records, já no DOM escondido). O número do time é o RITM; o seletor restrito evita pegar RITMs citados na descrição.

O Incidente do Backup também cria um **CS**, e o **INC** fica na aba Related Records, escondida. Até a 1.10.5 o seletor vazio lia só o texto visível (`inner_text`) e o robô aceitava o CS depois de 10 s. Desde a 1.10.6 o seletor é `body` (lê também o escondido) e o robô espera o INC por até 60 s; sem INC nesse tempo, aceita o CS (o chamado já foi criado). O número que o time usa no Backup é o **INC**.

Link de tratamento

`link_tratamento(cfg, numero)` monta `instancia_url + link_tratamento` com a tabela pelo prefixo: RITM → `sc_req_item`, INC → `incident`, REQ → `sc_request`, CS → `sn_customerservice_case`. Resultado: `/now/nav/ui/classic/params/target/sc_req_item.do%3Fsysparm_query%3Dnumber%3DRITM0421153`. Vai no `url` do retorno, do histórico e do painel.

Checagem de duplicado no portal (desligada)

`checar_duplicado_servicenow.ativo` (padrão `false`): antes de abrir, visita `url_busca` com `{id_curto}` e, se achar `regex_numero` na página, usa esse número. Fica desligada porque só um robô fica ativo e o histórico local já garante a idempotência.

7 Idempotência: nunca dois chamados

A garantia vem de três camadas: reserva atômica na fila, histórico local gravado em disco antes do clique, e recuperação conservadora depois de qualquer queda.

Histórico local (`processados.jsonl`)

`fila.Historico`: uma linha JSON por mudança de status; vale a última linha de cada `id`. Cada gravação faz `flush` + `os.fsync`.

STATUS	QUANDO	EFEITO NUM ARQUIVO COM O MESMO ID
`enviando`	Imediatamente antes do clique em Enviar (ou da conferência)	Bloqueia: vai para `/erro` "confira no ServiceNow"
`ok`	Número capturado (ou ignorado/manual)	Concluído sem chamado e sem Teams (repetido)
`erro`	Erro final com id conhecido	Pode ser reprocessado (move de volta para `pendentes`)

- **GMUD:** `id` = número da CHG. A mesma CHG nunca vira dois RITMs, mesmo com e-mail repetido ou dois fluxos ligados.
- **Backup:** `id` = `internetMessageId#<linha inteira do CSV>` (a linha tem o JobID, único).
- **Limite conhecido:** o histórico é **por máquina**. Se o mesmo alerta for reprocessado em outra máquina depois de `ok`, ela não sabe. Mitigação: só um robô ativo + MoveTo atômico (um arquivo só é reservado uma vez).

Recuperação depois de queda

SITUAÇÃO	QUEM TRATA	AÇÃO
Arquivo desta máquina em `processando`, histórico `ok`	`recuperar_reservas`	Conclui; grava retorno **só se o histórico tem número** (1.10.5: ignorado e "aberto à mão" não geram ✅ vazio no Teams).
Idem, histórico `enviando`	`recuperar_reservas`	`/erro` "interrompido depois de clicar em Enviar".
Idem, sem histórico ou `erro`	`recuperar_reservas`	Devolve a `pendentes`.
Arquivo de **outra** máquina parado há > 60 min	`tratar_reservas_abandonadas`	`/erro` "a máquina X reservou e não terminou".
Chamado aberto, falha ao gravar o resultado na fila	próximo ciclo	O histórico já tem `ok`: a recuperação conclui.

Reprocessar de propósito

Mover o JSON de `/erro` para `pendentes` reprocessa se o histórico não for `enviando`/`ok`. Para forçar (depois de conferir no ServiceNow que o chamado não existe), acrescente `"forcar_reprocessamento": true` no JSON: ignora histórico, cliente ignorado, pergunta de alerta antigo e checagem de duplicado.

8 Fila no SharePoint (API REST)

`FilaSharePoint` tem a mesma interface da `Fila` de pasta. Fala direto com `{site}/_api/...` usando `urllib`, autenticada pelos cookies da sessão Microsoft do perfil do navegador do robô. Sem OneDrive, sem senha, sem app registrado no Azure.

> **Onde fica a fila**
>
> Site `https://camara1.sharepoint.com/sites/MANAGEDSERVICE`, biblioteca "Documentos Compartilhados", pasta `YDUQS/N1/DispatchBot-Fila`. No config: `fila_sharepoint.site` e `fila_sharepoint.pasta` (caminho relativo ao servidor, com espaços).

Autenticação

- `obter_cookies(renovar)` = `Navegador.cookies_sharepoint`. Sem renovar: lê os cookies do perfil (`FedAuth` / `rtFa`); na maioria das vezes nem abre o site.
- Renovar: visita o site **sem janela**; se a Microsoft pedir login (ou o Edge sem janela der `net::ERR_FAILED`), reabre **com janela** e espera o plantonista (ícone amarelo). Pausado não abre janela.
- Gravações (POST) levam `X-RequestDigest` de `contextinfo`, guardado até 2 min antes de vencer (~30 min).
- Respostas 301/302/401/403 = sessão vencida: renova no máximo uma vez a cada 2 min e repete a chamada. Redirecionamentos não são seguidos (`_SemRedirecionar`). Persistindo: `PastaIndisponivel` "confira se a conta tem acesso ao site".
- Chamadas de outras threads (parada remota, tela de histórico) usam `pode_renovar=False`: nunca mexem no navegador.

Chamadas usadas

OPERAÇÃO	ENDPOINT
Listar arquivos	`GET web/GetFolderByServerRelativePath(decodedurl='…')/Files?$select=Name,ServerRelativeUrl,TimeLastModified`
Listar subpastas	`GET …/Folders?$select=Name`
Criar subpasta	`POST web/folders/AddUsingPath(decodedurl='…')`
Ler arquivo	`GET web/GetFileByServerRelativePath(decodedurl='…')/$value`
Gravar (sobrescreve)	`POST …/Files/AddUsingPath(decodedurl='nome',overwrite=true)`
Mover (reserva)	`POST …/MoveTo(newurl=@d,flags=0|1)?@d='destino'` — `flags=0` falha se o destino existe
Apagar	`POST …` com `X-HTTP-Method: DELETE`, `IF-MATCH: *` (404 é ignorado)
Digest	`POST contextinfo`

Caminhos vão em `decodedurl='…'` com aspas simples dobradas e o resto codificado (`_caminho`). Aceita JSON com ou sem BOM (`utf-8-sig`).

Reserva e concorrência

- **Reservar** = `MoveTo` de `pendentes/A.json` para `processando/A__HOST.json`. É atômico no servidor: se outra máquina já moveu, dá 404 e esta desiste. Depois o conteúdo é regravado para a data do arquivo virar a hora da reserva.
- `confirmar_reserva` sempre `True` no SharePoint (a janela de espera `janela_confirmacao_seg` só existe no modo de pasta sincronizada, com desempate pelo menor hostname).
- **Devolver** = MoveTo de volta com sobrescrita.
- **Concluir** = grava em `processados`, apaga de `processando`, grava o retorno. **Erro** = grava em `erro`, apaga, grava o retorno (JSON ilegível: `.erro.txt` primeiro e depois move, para refazer se a conexão cair no meio).

Cadências

O QUÊ	FREQUÊNCIA
Listar `pendentes`	Cada ciclo (15 s ou ao acordar)
`verificar_pasta` (subpastas)	5 min
Gravar `status/HOST.json`	5 min ou na troca de status/pausa
Ler `status` dos outros	No máx. 1 vez por minuto (cache); considera sinal de até 12 min
Pedido de plantão para esta máquina	30 s (no loop e entre alertas)
Arquivo `PARAR`	10 s (`intervalo_parar_seg`)

Modo antigo: pasta sincronizada (reserva)

Com `fila_sharepoint.site` vazio, `Fila` usa uma pasta local sincronizada pelo OneDrive (`pasta_sincronizada`). `pasta.py` acha a pasta pelo registro (`SyncEngines\Providers\OneDrive`), pode abrir um link `odopen://` (`link_sincronizar`) e fixa a pasta como "sempre no dispositivo". A reserva é `os.rename` + espera de `janela_confirmacao_seg` e desempate pelo menor hostname. Fica só como alternativa.

9 Navegador, sessão e login SSO

`Navegador` abre o Edge (ou o Chrome, pela ordem de `navegador.canais`) instalado na máquina, com `launch_persistent_context` no perfil próprio `%LOCALAPPDATA%\DispatchBot\browser-profile`. Nenhum navegador é empacotado.

Abertura

- Argumentos: `--start-maximized --no-first-run --no-default-browser-check --hide-crash-restore-bubble`, `no_viewport`, timeout padrão das ações `timeout_acao_seg` (30 s).
- Falhou nos dois canais: `matar_navegadores_orfaos()` (PowerShell/CIM: só processos `msedge`/`chrome` cuja linha de comando contém o perfil do robô; o navegador pessoal não é tocado) e tenta de novo.
- Fechamento pela pessoa: evento `close` marca como fechado; reabre no próximo uso.

Sessão do ServiceNow

- A sessão **não sobrevive** ao fechamento do navegador (confirmado em 01/10/2026). Por isso o padrão é `fechar_apos_ocioso_min: 0` (sempre aberto) e o robô abre o portal ao iniciar (`login_inicial`), para o login acontecer no começo do plantão. Na 1ª vez grava a flag `login_inicial_ok`.
- `manter_sessao`: navegador parado há `manter_sessao_min` (10) → recarrega o portal. Se a sessão caiu, pede o login na hora. Login não feito → tenta abrir de novo em 15 min.

Detecção da tela de login (`na_tela_de_login`)

1. URL contém algum de `login.padroes_url_login` (`login.microsoftonline.com`, `/login.do`, `external_login`, `/saml`...), ou
2. o host é diferente da instância, ou
3. algum de `login.seletores_tela_login` está visível (o Service Portal mostra o login no mesmo endereço: `input[type=password]`, "Use external login"...).

Espera do login (`esperar_login`)

- Clica em `login.botao_sso` ("Use external login") no máximo 2 vezes, com 20 s entre cliques.
- Na tela "External login" (`tela_email`): se `email_sso` estiver preenchido, digita o e-mail; clica em Submit (`botao_email`) no máximo 2 vezes, com 15 s entre cliques. Nunca digita senha.
- Carência de `aviso_apos_seg` (15 s) antes de avisar: o SSO costuma concluir sozinho. Depois: ícone amarelo, notificação, janelinha com "Mostrar janela de login".
- Concluído = 3 verificações seguidas (~6 s) fora da tela de login e com o portal pronto (`seletor_portal_pronto`, opcional). Evita falso positivo no meio dos redirecionamentos.
- Limite `timeout_login_min` (15) → `LoginNaoConcluido`. Pausa durante a espera → `Interrompido`.
- `ir_para(url)`: se passou pelo SSO e caiu em outra página (o SSO às vezes volta para a página inicial), navega de novo para a URL pedida; se cair no login outra vez → `LoginNaoConcluido`.

> **Fluxo real de login (Cirion)**
>
> Portal → "Use external login" (robô) → "External login" + Submit (robô, campo vazio) → conta Microsoft → Windows Hello (plantonista). Confirmado no portal real em 13/10/2026.

10 Plantão entre máquinas e histórico do time

Várias máquinas têm o robô instalado; só uma deve ficar ativa. O SharePoint serve de canal entre elas: status, pedidos de plantão e histórico. Não há trava rígida: é coordenação com aviso.

Nome do plantonista

`nome_plantonista` no config (máx. 40 caracteres). Vazio = usuário do Windows arrumado (`vinicius.morais` → "Vinicius Morais"). Perguntado pela janelinha do painel sempre que estiver vazio (uma vez por execução; fechar a janela grava o nome sugerido); trocado pelo menu "Meu nome no DispatchBot...". Vai no status, nos eventos e no painel dos colegas.

Outros robôs ativos

- `status_outros(12 min)` lê `status/*.json` das outras máquinas (também com o robô pausado) e `outros_ativos()` filtra os não pausados.
- Painel: faixa amarela "Também ativo: Fulano (HOST), ativo desde, último sinal há X min" + botão **Assumir o plantão**.
- Ativo com outro ativo: notificação no máximo 1 vez por hora por máquina.
- Retomar com outro ativo: caixa Sim (assumir) / Não (ativar mesmo assim) / Cancelar (continua pausado).

Assumir o plantão

1. Quem assume grava `plantao/<HOST alvo>.json` (`{de_nome, de_host, em, ts}`) e se ativa. Evento `assumiu`.
2. O robô alvo lê o pedido a cada 30 s, no loop e **entre um alerta e outro** (nunca no meio), apaga o arquivo, pausa com o detalhe "Fulano assumiu o plantão às HH:MM", notifica e mostra uma caixa ao dono. Evento `plantao_passado`. Como em qualquer pausa, o loop fecha o navegador do robô no ciclo seguinte (`fechar_navegador`).
3. Pedido com mais de 15 min (máquina estava desligada) ou alvo já pausado: só apaga.
4. Quem assumiu é avisado quando o status do outro aparece pausado, ou se ele não responder em 12 min. Se o outro simplesmente parar de dar sinal (status com mais de 12 min), o pedido é esquecido só com uma linha no log, sem notificação.

Histórico do time

- Arquivo por máquina e dia: `historico/<AAAA-MM-DD>_<HOST>.jsonl`. O SharePoint não faz append: o robô guarda as linhas do dia em memória e **regrava o arquivo inteiro**; ao reabrir no mesmo dia, lê o que já existe e continua. Só a própria máquina grava no seu arquivo (sem conflito).
- `Robo.evento()` enfileira; `gravar_eventos` (thread do robô) grava por dia; falhou, tenta no próximo ciclo (até 500 eventos em memória).
- Tela **Histórico do time** (rodapé do painel): Hoje/Ontem, junta todas as máquinas e os eventos ainda não gravados; a leitura roda numa thread à parte com `pode_renovar=False`.

EVENTO	QUANDO
`iniciou` / `encerrou`	Robô aberto / fechado

`ativou` / `pausou`	Pausa pelo usuário (menu/painel)
`emergencia`	Parada pelo menu, atalho ou arquivo PARAR
`assumiu` / `plantao_passado`	Pedido de plantão enviado / recebido
`chamado`	Chamado aberto (com tipo, numero, url)
`erro`	Alerta foi para `/erro`
`manual` / `ignorado`	Alerta antigo marcado como aberto à mão / cliente ignorado
`login` / `login_ok`	Começou a esperar login / login feito
`envio`	Envio automático ligado/desligado
`nome`	Troca de nome

11 Parada de emergência

Para o caso de o robô abrir chamados errados. Interrompe no meio do passo, sem esperar o alerta terminar.

ORIGEM	COMO	ALCANCE	COMO LIBERAR
`menu`	"PARAR AGORA" no menu do ícone (pede confirmação desde a 1.7.1) ou no painel (também pede confirmação)	Esta máquina	Desmarcar Pausado
`atalho`	1º livre de `atalho_emergencia`: `Ctrl+Alt+Shift+P`, `Ctrl+Alt+Shift+X`, `Ctrl+Alt+Pause`	Esta máquina	Desmarcar Pausado
`remota`	Arquivo `PARAR*` na raiz da `DispatchBot-Fila` (1ª linha = motivo)	Todas as máquinas, em até 10 s	Apagar o arquivo

O que acontece (`parar_agora`)

1. Sinaliza o evento `emergencia` (todas as threads veem) e registra o evento no histórico.
2. Parada local: grava `pausado: true` e `emergencia: {origem, em}` no config. Nem reiniciar nem o vigia fazem o robô voltar sozinho.
3. Ícone vermelho com o motivo + notificação.
4. `matar_navegadores_orfaos()`: derruba o navegador do robô, então a ação em andamento falha na hora.
5. O alerta em andamento: sem `enviando` volta intacto para `pendentes`; com `enviando` vai para `/erro` "confira no ServiceNow".

A parada remota não grava no config: some sozinha quando o arquivo é apagado. Enquanto o PARAR existir, "Retomar" pelo menu é recusado com aviso.

12 Interface: bandeja, painel e janelinha

Ícone da bandeja (`bandeja.py`)

Avatar (`assets/avatar.png`) com bolinha de status no canto inferior direito. Tooltip `DispatchBot - <status>: <detalhe>` (máx. 127 caracteres).

STATUS	COR	QUANDO
`ativo`	verde `#2E9E5B`	Lendo a fila / pronto
`preparando`	azul `#2F7FD1`	Só no modo pasta: esperando o OneDrive
`login`	amarelo `#E0A100`	Esperando login (ServiceNow ou SharePoint)
`confirmar`	amarelo	Formulário preenchido esperando conferência
`erro`	vermelho `#D23C3C`	Erro de infraestrutura, alerta em `/erro` ou emergência
`pausado`	cinza `#8A8F8C`	Pausado (detalhe: quem assumiu o plantão)

Menu: PARAR AGORA (com o atalho ativo ao lado) · Status · Versão · **Abrir painel** (clique simples no ícone) · Verificar pendentes agora · Pausado · Enviar GMUD sem conferir · Enviar Backup sem conferir · Meu nome no DispatchBot... · Abrir pasta DispatchBot-Fila · Abrir pasta de logs · Abrir config.json · Sair.

Painel (`painel.py`)

- CustomTkinter em tema escuro, ícones do Windows (fontes Segoe Fluent Icons/MDL2 desenhadas em imagem). Thread própria; fechar a janela só esconde.
- Cabeçalho (avatar, status, detalhe, resumo "plantonista: nome", Pausar/Retomar), faixa "Também ativo", 4 contadores do dia, "Precisa de você", "Chamados abertos hoje" (Tratar/Copiar), "Erros de hoje: abrir à mão" (motivo, Abrir no ServiceNow, Copiar), rodapé (Verificar agora, Logs, Histórico do time, PARAR AGORA).
- Fonte das listas: `painel.jsonl` local (só as últimas 2000 linhas, filtradas pelo dia). Atualiza a cada 3 s; a seção "Precisa de você" e as contagens regressivas a cada 1 s.
- O painel não altera fila nem config diretamente: chama os callbacks do `Robo`.

Janelinha "Precisa de você"

Toplevel no canto inferior direito, sempre por cima, com rolagem (altura = conteúdo, até ~65% da tela). Abre sozinha mesmo com o painel fechado. Mostra: login esperando ("Mostrar janela de login"), conferência (campos, motivo "Não enviei sozinho: ...", contagem, "Mostrar formulário", "Pular este alerta") e decisões de alertas antigos (Abrir chamado / Já foi aberto à mão / Decidir depois). Sem o painel disponível, a reserva são caixas do Windows (`MessageBoxW`).

13 Configuração (`config.json`)

Fica em `%LOCALAPPDATA%\DispatchBot\config.json`. Na 1ª execução é copiado de `config.default.json` (empacotado). Relido a cada ciclo. Chaves que começam com `_` são ajuda e são ignoradas.

Carga, mescla e gravação

- `carregar()` lê com `utf-8-sig`; JSON inválido → `ConfigInvalido` com linha e coluna (o ciclo inteiro é pulado: o robô fica vermelho com a linha do erro e não lê a fila até o arquivo ser corrigido; ações do menu que gravam o config, como Pausado, envio e nome, também falham).
- **Mescla** (`_mesclar`): chaves que existem no modelo e não no arquivo do usuário entram sozinhas, recursivamente em dicionários. Valores do usuário sempre prevalecem. **Listas não são mescladas**: um passo novo em `passos` não chega a um config já instalado.
- Exceções pontuais: `botao_enviar` vazio no usuário e preenchido no modelo é copiado do modelo; `captura_numero` com `seletor` vazio é trocada pela do modelo (1.10.6, captura do INC); clientes do modelo que faltam em `ignorar_clientes` são acrescentados à lista instalada (1.10.5; não dá para tirar um cliente do modelo só pelo config.json).
- `salvar()`: grava em `.tmp` e faz `os.replace` (atômico).

Chaves gerais

CHAVE	PADRÃO	USO
`instancia_url`	https://envdatacenter.service-now.com	Base de todas as URLs relativas.
`portal_path`	`/esc`	Portal aberto no login e no "manter sessão".
`link_tratamento`	`/now/nav/ui/classic/…{tabela}…{numero}`	Link do chamado na interface clássica.
`fila_sharepoint.site` / `.pasta`	MANAGEDSERVICE / .../DispatchBot-Fila	Fila no SharePoint. `site` vazio = modo pasta.
`fila_sharepoint.intervalo_parar_seg`	10	Busca do arquivo PARAR.
`pasta_sincronizada`, `link_sincronizar`	""	Só modo pasta (OneDrive).
`intervalo_polling_seg`	15	Intervalo do loop.
`idade_minima_arquivo_seg`	5	Ignora arquivos recém-criados.
`janela_confirmacao_seg`	10	Só modo pasta: espera antes de confirmar a reserva.
`pausado`	false	Gravado pelo menu/painel.
`emergencia`	(ausente)	Gravado pela parada local; removido ao retomar.

`nome_plantonista`	""	Nome visto pelo time.
`envio_automatico.GMUD` / `.BACKUP`	true / true	Envio sem conferência (seção 6).
`perguntar_alerta_antigo_min`	120	Idade para perguntar antes de abrir. 0 = nunca pergunta.
`tentativas_por_alerta`	3	Tentativas em falha passageira.
`espera_entre_tentativas_seg`	20	Multiplicado pelo número da tentativa.
`max_espera_login_min`	60	Alerta esperando login há mais que isso vai para `/erro`.
`reserva_abandonada_min`	60	Reserva de outra máquina parada → `/erro`.
`atalho_emergencia`	3 combinações	Tentadas em ordem.
`navegador.canais`	msedge, chrome	Ordem de tentativa.
`navegador.timeout_acao_seg`	30	Timeout padrão do Playwright.
`navegador.timeout_login_min`	15	Espera máxima de um login.
`navegador.fechar_apos_ocioso_min`	0	0 = sempre aberto (recomendado).
`navegador.manter_sessao_min`	10	Recarga do portal com o robô parado. 0 = desliga.
`login.*`	ver seção 9	`padroes_url_login`, `seletores_tela_login`, `botao_sso`, `email_sso`, `tela_email`, `campo_email`, `botao_email`, `aviso_apos_seg`, `seletor_portal_pronto`.
`checar_duplicado_servicenow.*`	ativo: false	Busca opcional no portal (seção 6).

Bloco de um tipo (`tipos.<NOME>`)

CHAVE	USO
`herda`	Usa outro bloco como base. Dicionários (ex.: `variaveis`) são somados; as outras chaves são substituídas. `tipo_efetivo()` aplica.
`url_formulario`	URL do formulário sem o domínio (aceita placeholders).
`seletor_formulario_pronto`	Espera esse elemento antes do 1º passo.
`variaveis` / `padroes`	Fixas / sobrescrevíveis pelo alerta.

`validar`	Regex por campo do alerta (casamento inteiro).
`passos`	Lista de `{campo, acao, seletor, valor, busca, seletor_busca, seletor_opcao, mapa, opcional, pular_se_vazio}`.
`botao_enviar`	Seletor do Submit/Enviar (`#submit-btn` nos dois formulários).
`seletor_erro_validacao`	Mensagem de recusa do ServiceNow (vazio nos dois formulários).
`confirmacao_timeout_min`	Prazo da conferência (30).
`captura_numero`	`regex`, `preferir`, `seletor`, `espera_preferido_seg`, `timeout_seg`.
`ignorar_clientes`	Só Backup: hosts cujas falhas não abrem chamado.

Tipos configurados

TIPO	FORMULÁRIO	TÍTULO	OBSERVAÇÃO
`_GMUD` (modelo)	Hosting `sys_id=42761d80…3593` (POP 11)	`#GMUD {chg} | {time_titulo} | {data_execucao| dd/MM HH:mm} |`	Conta `…(3-9R2V6QXWXV)`, CallerID `OPERACAO YDUQS`, Categoria `Other`, descrição = `{descricao}`; `validar.chg = CHG\d{7}`.
`INFRA` / `DBA` / `SO`	herda `_GMUD`	`SOC` / `DBA` / `SO`	Em produção.
`GMUD_BACKUP`	herda `_GMUD`	`BACKUP`	Fila `SN_TI_CHG_EXECUTOR_Backup_Cirion` (desde 16/10/2026). Não confundir com `BACKUP`.
`PRODUCAO`	herda `_GMUD`	`PREENCHER`	Fora do escopo: sempre `/erro` (`TipoNaoMapeado`).
`BACKUP`	Incidente, cartão Backup `sys_id=ca634cdd…35b7`	`#FALHA DE BACKUP {titulo}`	Account, Contact, Service Affected (Veritas Netbackup), Failure Type (Execution failure), Impacto (mínimo), Short Description, Description. `ignorar_clientes`: `appflexrj01-inst01.smpdc.gblx`, `appnburj03`, `nbmasterbrarj01`.

Tipo novo sem código

Crie um bloco em `tipos` (com `"herda": "_GMUD"` se for o mesmo formulário), mapeie os seletores com o Mapear, teste com `--testar` e ajuste o fluxo de entrada para gravar o `tipo` e o Retorno para ter o Case do chat. Se o tipo for de GMUD, o grupo de envio automático já é `GMUD`.

14 Contratos de dados (JSON)

Todos os arquivos são UTF-8 (com ou sem BOM na leitura; sem BOM na gravação), um objeto JSON por arquivo, exceto os `.jsonl` (uma linha por registro).

Entrada · `pendentes/<TIPO>_<aaaaMMdd_HHmmss>_<guid8>.json`

```
// GMUD (fluxo "GMUD | Todos")
{
  "versao": 2,
  "id": "CHG0048707",
  "tipo": "SO",
  "grupo_executor":
"SN_TI_CHG_EXECUTOR_CIRION_SO",
  "chg": "CHG0048707",
  "ctask": "CTASK…",
  "data_execucao": "16/10/2026 14:00",
  "tipo_mudanca": "Normal",
  "titulo": "…",
  "descricao": "…",
  "recebido_em": "2026-10-16T12:05:00-03:00",
  "dados_brutos": "…"
}
```

```
// Falha de Backup (BackupBot)
{
  "versao": 2,
  "id": "<msgid@…>#26661937,NBMASTER…,

POLITICA,cliente,UNKNOWN,…,Failed,4287",
  "tipo": "BACKUP",
  "titulo": "POLITICA | cliente | Job
26661937
             | Status 4287",
  "descricao": "…",
  "servico_afetado": "(opcional)",
  "failure_type": "(opcional)",
  "recebido_em": "2026-10-16T08:24:00-03:00",
  "dados_brutos": "…"
}
```

CAMPO	OBRIGATÓRIO	USO NO ROBÔ
`id`	sim	Chave de idempotência. GMUD = CHG.
`tipo`	sim*	Bloco do config. *Sem ele, deduzido de `grupo_executor`.
`chg`, `data_execucao`, `descricao`	GMUD	Título e descrição; `chg` validada.
`titulo`, `descricao`	Backup	Short Description e Description; cliente lido do título/`id`.
`recebido_em`	recomendado	Alerta antigo e limite de espera do login (ISO 8601).
`forcar_reprocessamento`	não	`true` ignora as proteções (seção 7).
`teams`, `dados_brutos`, `ctask`, `tipo_mudanca`, `versao`	não	Repassados/ignorados; disponíveis como placeholders.

Retorno · `retorno/<nome original>.json`

```
{
  "id": "CHG0048707", "tipo": "SO", "titulo": "…",
  "status": "ok",                  // ok | erro
  "numero": "RITM0421153",         // vazio no erro
  "url": "https://envdatacenter.service-
now.com/now/nav/ui/classic/params/target/sc_req_item.do%3Fsysparm_query%3Dnumber%3DRITM0421153",
  "mensagem": "",                  // motivo no erro
  "maquina": "HOST", "processado_em": "2026-10-16T12:07:31-03:00", "teams": {}
}
```

Arquivos de coordenação

ARQUIVO	CAMPOS
`status/<HOST>.json`	`status`, `pausado`, `versao`, `usuario`, `nome`, `ativo_desde`, `detalhe`, `host`, `atualizado_em`, `ts` (epoch). Ao sair: `status: "encerrado"`.
`plantao/<HOST alvo>.json`	`de_nome`, `de_host`, `em`, `ts`
`historico/<dia>_<HOST>.jsonl`	`em`, `host`, `nome`, `usuario`, `evento`, `texto` + extras (`tipo`, `numero`, `url`, `de_host`, `de_nome`, `para_host`, `para_nome`)
`processados/…`	Alerta + `resultado`: `numero`, `url`, `maquina`, `processado_em`
`erro/…`	Alerta + `erro`: `mensagem`, `maquina`, `em`

Arquivos locais

ARQUIVO	LINHA
`processados.jsonl`	`{id, status, em, maquina, tipo, numero?, url?, mensagem?, ignorado?, manual?}` (`ignorado` = nome do cliente que casou)
`painel.jsonl`	`{em, status: ok|erro, tipo, alerta (CHG/título), numero, url, mensagem}`

15 Fluxos do Power Automate

Três fluxos de nuvem, agrupados na Solução "DispatchBot". Pacotes exportados em `fluxos/`; passo a passo completo em `docs/power-automate.md`.

ENTRADA · PRODUÇÃO DESDE 05/10/2026

GMUD | Todos

- Gatilho: e-mail novo de `noreply.yduqs@service-now.com`, assunto "Uma tarefa de Mudança foi atribuída". Trigger condition: corpo contém `SN_TI_CHG_EXECUTOR_CIRION_` **ou** (minúsculas) `sn_ti_chg_executor_backup_cirion`.
- `Html to text` → Compose `Time` (INFRA/DBA/SO/PRODUCAO pelo grupo executor; `GMUD_BACKUP` quando contém `backup_cirion`) → `Recorte` (entre "Prezado" e "Informações sobre sua") → Switch por `Time` posta o card no chat do time (Default: Terminate) → `JSON GMUD` → Create file `<Time>_….json` em `pendentes`.
- CHG por `indexOf('CHG0')`; data depois de `Início:`; descrição = Descrição resumida.

ENTRADA

BackupBot

- Gatilho na caixa do Backup (outra caixa, por isso não há fluxo de entrada único). Lê o CSV anexo (`base64ToString(contentBytes)` do **primeiro anexo**), filtra as falhas, posta o card e, no loop "Para Cada Falha", monta `JSON_BACKUP` e cria o arquivo em `pendentes`.
- CSV sem cabeçalho: `[0]` JobID, `[1]` Master, `[2]` Política, `[3]` Cliente, `[4]` Agendamento, datas entre aspas **com vírgula dentro**, Status, Código. O `split` só é confiável até `[4]`; o Código vem de `last(...)`.
- Título: `take(Política | Cliente | Job <id> | Status <código>, 130)`.

SAÍDA · DESDE 09/10/2026

DispatchBot | Retorno

- Gatilho: arquivo criado em `…/DispatchBot-Fila/retorno` (trigger condition: termina em `.json`) → Get file content → Compose `Retorno` = `json(string(body('Get_file_content')))` → Compose `Mensagem` (✅ ou ⚠️; no BACKUP usa o `titulo`, nos outros o `id`) → Switch em `outputs('Retorno')?['tipo']`.

CASE	CHAT EM GRUPO (FLOW BOT)
`INFRA`	(FC) NET / SOC
`DBA`	(FC) BANCO DE DADOS (DBA'S + N1 + Gestão)
`SO`	(FC) LINUX - WINDOWS - VMWARE
`BACKUP`, `GMUD_BACKUP`	(FC) NETBACKUP
`TESTE`	Automação DispatchBot (testes)

Cuidados

- Arquivo de teste do Retorno vai em `/retorno`, nunca em `/pendentes`.
- Chats em grupo não têm thread: o Retorno posta mensagem nova no mesmo chat do card.
- Risco aceito: o BackupBot usa o primeiro anexo; se não for o CSV, o texto sai ilegível e a guarda do robô manda para `/erro`.
- Regra do projeto: não alterar fluxos em produção sem necessidade; preferir criar novos (exceção registrada: Case `GMUD_BACKUP`).

16 Tratamento de erros

Regra geral: erro de **dado** ou de **formulário** vai para `/erro` e o Teams avisa (o plantonista abre à mão); erro de **infraestrutura** deixa o alerta na fila e o ícone vermelho até passar.

SITUAÇÃO	COMPORTAMENTO
Rede, instância lenta, navegador travado	Até 3 tentativas com navegador novo; depois `/erro`.
Formulário recusado pelo ServiceNow	`/erro` com a mensagem da tela (exige `seletor_erro_validacao`).
Falha depois do clique em Enviar	`/erro` "confira no ServiceNow"; nunca reenvia.
Envio automático com obrigatório vazio	Não envia; conferência pelo plantonista com o motivo.
Ninguém faz login	Alerta volta a `pendentes`; depois de 60 min desde o recebimento → `/erro` ⚠️.
SharePoint fora do ar / sem acesso	Vermelho (`PastaIndisponivel`); volta sozinho.
Sessão Microsoft vencida	Renova sozinho; se pedir login, janela + amarelo.
`config.json` com erro de digitação	Vermelho com a linha do erro; não processa nada até corrigir o arquivo.
Alerta antigo	Pergunta; sem resposta, espera em `pendentes` sem travar os novos.
Texto ilegível	`/erro` ⚠️ "abra à mão pelo e-mail original".
Janela do Edge fechada	Notifica e reabre no próximo ciclo (não fica vermelho).
Robô fechado no meio de um alerta	Recuperação na volta (seção 7).
Máquina de colega caiu com alerta reservado	Depois de 60 min → `/erro` para conferência.
Edge preso no perfil do robô	Mata só os processos do perfil do robô e reabre.
Exceção inesperada na thread do robô	Supervisor recria a thread em até 30 s; o vigia reabre o processo se ele morrer.

Mensagens amigáveis

`MENSAGENS_AMIGAVEIS` troca erros técnicos por texto para o plantonista (janela fechada, `Timeout`, `net::ERR_INTERNET_DISCONNECTED`, `ERR_NAME_NOT_RESOLVED`, `getaddrinfo failed`...). Para os demais, usa a 1ª linha da exceção (até 500 caracteres). O log mantém o traceback completo.

17 Arquivos locais, logs e diagnóstico

CAMINHO	CONTEÚDO
`%LOCALAPPDATA%\Programs\DispatchBot\`	Binários (onedir do PyInstaller), `exemplos\*.json`, desinstalador.
`%LOCALAPPDATA%\DispatchBot\config.json`	Configuração do usuário.
`…\logs\robo.log`	Log rotativo: 2 MB × 10 arquivos, nível INFO, formato `data hora NÍVEL mensagem`.
`…\logs\screenshots\`	PNG de página inteira nos erros (`AAAAmmdd_HHMMSS_erro_<tipo>_<arquivo>.png`); apagados após 30 dias.
`…\processados.jsonl`	Histórico de idempotência (não apagar).
`…\painel.jsonl`	Resultados para o painel.
`…\browser-profile\`	Perfil do Edge do robô (cookies do ServiceNow e da Microsoft).
`…\campos_mapeados.json`	Saída do Mapear.
`…\encerrado_pelo_usuario`	Flag: o vigia não reabre.
`…\login_inicial_ok`	Flag: login inicial já feito.

Mensagens do log que valem saber

TRECHO	SIGNIFICA
`Robô 1.10.6 iniciado em HOST.`	Thread do robô subiu.
`Processando <arquivo>`	Reserva feita.
`Envio automático suspenso (...)`	Caiu para a conferência; o motivo vem entre parênteses.
`Chamado RITM… aberto (SO, …).`	Sucesso.
`-> /erro: …`	Erro final com motivo.
`Sessão do SharePoint vencida (HTTP 401); renovando.`	Renovação normal.
`Login no ServiceNow necessário. Aguardando…`	Passou a carência; plantonista avisado.
`Tela External login: clicando em Submit.`	Passo automático do SSO.
`Alerta já processado (chamado …). Arquivo repetido ignorado`	Idempotência atuou.
`PARADA DE EMERGÊNCIA (origem)`	Parada acionada.
`A thread do robô parou inesperadamente; reiniciando.`	Supervisor atuou; veja o traceback acima.

Roteiro de diagnóstico

1. **Ícone e painel:** status, detalhe, "Precisa de você", erros de hoje com motivo.
2. **Fila:** onde está o arquivo (`pendentes` parado = robô pausado/login; `processando` = reservado por qual HOST; `erro` = motivo no JSON).
3. **status/**: qual máquina está ativa, versão, último sinal.
4. **Log** no horário + screenshot correspondente.
5. **processados.jsonl**: último status do `id` (`enviando` explica por que não reenviou).
6. **Histórico do time**: quem pausou, assumiu, ligou/desligou envio.

18 Build, instalador e Releases

Build local (`build\build.ps1`)

1. Cria `.venv` e instala `requirements.txt` (`playwright` , `pystray` , `pillow` , `customtkinter`) + `pyinstaller` . Não roda `playwright install` .
2. Lê a versão de `robo\versao.py` (fonte única).
3. `py_compile` de todos os `robo\*.py` .
4. `build\gerar_imagens.py` (ícone e imagens do instalador a partir do avatar), `build\versao_exe.py` (recurso de versão: nome "DispatchBot" nas notificações) e PyInstaller com `build\robo.spec` .
5. Assinatura opcional com `DISPATCHBOT_CERT_THUMBPRINT` (signtool do Windows SDK).
6. Inno Setup 6 (`installer\DispatchBot.iss`) → `dist\instalador\DispatchBot-Setup-<versão>.exe` .
7. Com `-Publicar` : exige tudo commitado, faz `git push` e cria/atualiza a Release `v<versão>` com o instalador, o manual e esta documentação (avisa se algum PDF estiver mais antigo que o HTML). Usa sempre notas automáticas (`--generate-notes`): `build\notas` só é lido pelo Actions. Sem o Inno Setup, o build só avisa e termina sem instalador.

`robo.spec` : dois executáveis numa pasta onedir — `DispatchBot.exe` (sem console, com ícone e recurso de versão) e `DispatchBot-Mapear.exe` (com console). Inclui o driver do Playwright e o CustomTkinter (`collect_all`), `config.default.json` e `assets/avatar.png` .

Release automática (GitHub Actions)

`.github/workflows/release.yml` : dispara ao enviar uma tag `v*` ou à mão (Run workflow). Em `windows-latest` : Python 3.12, Inno Setup (choco, se faltar), `build.ps1` , e publica a Release com os anexos **DispatchBot-Setup-<versão>.exe**, **Manual-DispatchBot.pdf** e **DispatchBot-Documentacao-Tecnica.pdf**. Antes de publicar, gera os dois PDFs de novo com o Edge (`--headless=new --disable-gpu --no-pdf-header-footer --print-to-pdf`) a partir do HTML da versão; se o PDF não sair, avisa e anexa o do repositório. Notas de `build\notas\<versão>.md` (sem o arquivo: notas automáticas). Se a Release já existe, só troca os anexos.

Instalador (Inno Setup)

- Por usuário, sem admin (`PrivilegesRequired=lowest`), em `{localappdata}\Programs\DispatchBot` ; textos para o plantonista; avisa se não houver Edge nem Chrome.
- Atualização limpa: remove `{app}\_internal` ; config, logs e sessão ficam em `%LOCALAPPDATA%\DispatchBot` .
- Antes de instalar/desinstalar: sinaliza o evento `DispatchBot_Sair` e espera o mutex sumir por até 30 s (o robô termina o alerta em andamento); depois `taskkill` .
- Autostart: `HKCU\…\Run\DispatchBot` . Vigia: tarefa agendada `DispatchBot Vigia` a cada 5 min (`schtasks /SC MINUTE /MO 5 /IT`) com `--vigia` .
- Desinstalação: remove a tarefa e pergunta se apaga também `%LOCALAPPDATA%\DispatchBot` .
- Modo silencioso: `/VERYSILENT` (o robô também é aberto no fim).
- Arquivos `.iss` e `.ps1` precisam ser UTF-8 **com BOM**.

Publicar uma versão

1. Subir `VERSAO` em `robo\versao.py` e escrever `build\notas\<versão>.md` .

2. Se mudou algo para o plantonista: atualizar `docs\manual\manual.html` e regerar o PDF. Se mudou algo técnico: atualizar `docs\tecnico\documentacao-tecnica.html` (versão no `@page` e na capa) e regerar o PDF.
3. Commit, tag `v<versão>` e push da tag (Actions publica) — ou `build.ps1 -Publicar` numa máquina Windows.

```
REM gerar os PDFs (Edge do Windows)
msedge --headless=new --no-pdf-header-footer --print-to-pdf="%CD%\docs\manual\Manual-
DispatchBot.pdf" "%CD%\docs\manual\manual.html"
msedge --headless=new --no-pdf-header-footer --print-to-pdf="%CD%\docs\tecnico\DispatchBot-
Documentacao-Tecnica.pdf" "%CD%\docs\tecnico\documentacao-tecnica.html"
```

19 Ferramentas: Mapear, --testar e Demo

Mapear campos (`DispatchBot-Mapear.exe`)

1. Pause o robô (os dois usam o mesmo perfil do navegador).
2. Menu Iniciar → DispatchBot → **Mapear campos do ServiceNow**: abre o portal com o Playwright Inspector.
3. Navegue até o formulário e clique em Resume (▶): o console lista seletor, ação sugerida e rótulo de cada campo; a lista completa (com opções dos selects) vai para `campos_mapeados.json`.
4. Copie a URL para `url_formulario` e os seletores para os `passos`.

Testar sem enviar

```
"%LOCALAPPDATA%\Programs\DispatchBot\DispatchBot-Mapear.exe" --testar
"%LOCALAPPDATA%\Programs\DispatchBot\exemplos\INFRA.json"
```

Preenche o formulário do tipo do JSON **sem enviar** (deixa passar `PREENCHER`), lista os campos obrigatórios marcando `VAZIO` os não preenchidos e os botões visíveis. Falha num passo: screenshot e a lista mesmo assim. `--enviar` abre o chamado de verdade (só com CHG fictícia e combinado com o time).

EXEMPLO (PASTA `EXEMPLOS`)	ESPERADO NA FILA REAL (COPIADO PARA `PENDENTES`)
`INFRA.json`, `DBA.json`, `SO.json`	Formulário Hosting preenchido (CHGs fictícias `CHG999000x`).
`BACKUP.json`	Incidente com os 6 obrigatórios preenchidos.
`GMUD_SEM_DATA.json`	`/erro` sem abrir (dado faltando).
`PRODUCAO.json`	`/erro` (tipo não mapeado).
`TIPO_INVALIDO.json`	`/erro` (tipo desconhecido).
`RETORNO_TESTE.json`, `RETORNO_TESTE_INFRA.json`	Em `/retorno`: `RETORNO_TESTE` posta no chat de teste; `RETORNO_TESTE_INFRA` cai no Case INFRA e posta no chat **real** (FC) NET / SOC (usar só combinado com o time).

Com `--testar` é diferente: nada vai para `/erro`. `PRODUCAO.json` preenche (o `PREENCHER` passa sem envio), `GMUD_SEM_DATA.json` mostra `FALHOU: …` e `TIPO_INVALIDO.json` mostra "Tipo ... não existe".

Demo

`Demo DispatchBot.cmd` → `robo\demo.py`: janela com dois botões que preenchem GMUD (INFRA) e Backup no portal real sem enviar. Serve para apresentar o robô.

20 Testes e manutenção

Ambiente de teste local

- **LOCALAPPDATA de teste:** rode com `LOCALAPPDATA` apontando para uma pasta temporária. Config, histórico, logs e perfil ficam lá; nada encosta na instalação real.
- **Fila local:** `fila_sharepoint.site` vazio e `pasta_sincronizada` numa pasta com as subpastas; jogue os exemplos em `pendentes`.
- **ServiceNow simulado:** `http.server` servindo um formulário com a mesma estrutura do Service Portal (`#s2id_sp_formfield_*`, select2 com `.select2-drop-active`, asterisco `.mandatory`, `#submit-btn` que mostra um número). Aponte `instancia_url` para ele.
- **Instalador:** instalar com dados de teste e desinstalar no fim; conferir que nada ficou no `%LOCALAPPDATA%\DispatchBot` real.

Nunca

- Abrir chamado real em teste (`--enviar` só combinado), nem executar o BackupBot.
- Colocar arquivo de teste em `/pendentes` da fila real (Retorno de teste vai em `/retorno` com `tipo: TESTE`).
- Simular teclado (SendKeys) na área de trabalho do usuário.

Checklist de uma mudança

1. Ler o trecho do código e o `CLAUDE.md` (decisões e estado).
2. Mudar mantendo o estilo: código simples, comentários e mensagens em português.
3. Testar com fila local e formulário simulado; para o painel, prints com dados de exemplo.
4. Atualizar versão, notas, README/manual/esta documentação conforme o que mudou.
5. Gerar o instalador, instalar por cima numa máquina, conferir o log e o ícone.

Pontos de extensão

QUERO...	ONDE
Novo tipo de alerta	Bloco em `tipos` + fluxo de entrada + Case no Retorno.
Novo campo num formulário	Passo em `passos`. Atenção: listas não são mescladas; máquinas instaladas precisam do passo no próprio `config.json`.
Nova regra para não abrir chamado	Guarda em `Robo.processar`, **antes** da abertura e depois da idempotência.
Nova ação de passo	`servicenow.executar_passo`.
Novo evento do histórico	`Robo.evento(nome_evento, texto, /, **extra)` + rótulo/cor em `painel.EVENTOS`. `nome_evento` é só posicional: os extras podem levar `tipo=` (tipo do alerta) sem conflito.
Nova mensagem amigável	`main.MENSAGENS_AMIGAVEIS`.

21 Decisões fechadas, limites e pendências

Decisões fechadas (não reabrir)

- Chamados pelo **Employee Center** (`/esc`), não pelo Service Operations Workspace.
- Se POPs divergirem, vale o mais recente; o BackupBot (PAD, 15/07/2026) vale sobre o POP 03.
- Depois de abrir a GMUD: **só avisar no Teams**; sem planilha CHANGES e sem Planner.
- Só o Dispatcher N1 da vez fica ativo; fila numa biblioteca do SharePoint do time.
- O robô não inventa dados; nunca reenvia sozinho depois do clique em Enviar.
- Número da GMUD = RITM e do Backup = INC (nunca o CS), com link de tratamento na interface clássica.
- Envio automático (GMUD e Backup) por padrão desde 16/10/2026, com conferência de obrigatórios.
- Login: robô clica em "Use external login" e "Submit" sem preencher nada; nunca senha.
- Fora do escopo: entrada por e-mail genérico/IA, fluxo de tratamento do chamado, PRODUCAO.

Limites conhecidos

- Idempotência por máquina (histórico local); depende de um robô ativo e da reserva atômica.
- Sessão do ServiceNow não sobrevive ao fechamento do navegador; o timeout real da Cirion não foi medido.
- Seletores do Service Portal podem mudar com atualização da Cirion: o envio automático cai para conferência (obrigatório vazio) ou o passo falha e vai para `/erro` com screenshot.
- BackupBot lê o primeiro anexo do e-mail (risco aceito; coberto pela guarda de texto ilegível).
- Sem certificado de assinatura de código: SmartScreen/antivírus podem reclamar (o antivírus já removeu um desinstalador instalado em `%TEMP%`).

Pendências (outubro de 2026)

- 1º Backup real que abra chamado (Submit → INC → Teams) e a 1ª mensagem ⚠️ real do Retorno.
- 1ª GMUD de Backup (`GMUD_BACKUP`) real.
- Teste de plantão com duas máquinas reais no SharePoint.
- Instalar na máquina do N1 e compartilhar o manual com o time.
- Recomendado: certificado de assinatura de código.

Linha do tempo resumida

VERSÃO	DESTAQUES
1.6.x	Fila direto no SharePoint (API REST); Failure Type no Backup; `--testar` lista obrigatórios.
1.7.x	Painel do plantonista; alerta antigo pergunta antes; PARAR do menu com confirmação; limpeza de `\0`.
1.8.x	Avatar com bolinha de status; janelinha "Precisa de você"; guarda de texto ilegível; Submit no External login.
1.9 / 1.10.0	Painel escuro CustomTkinter (versão A); mensagens amigáveis; janela do Edge fechada não é erro.

1.10.3	Janelinha com rolagem; saída garantida do processo.
1.10.4	Envio automático com conferência de obrigatórios; clientes ignorados no Backup; `GMUD_BACKUP`; plantão entre máquinas e histórico do time. 1ª GMUD real de ponta a ponta (16/10/2026).
1.10.5	Recuperação não gera ✅ sem número para alerta ignorado/aberto à mão; `nbmasterbrarj01` ignorado e comparação só pela coluna Cliente. Release automática no GitHub Actions.
1.10.6	Backup captura o **INC** (aba escondida) em vez do CS; `Robo.evento` não estoura mais com `tipo=` nos extras (o arquivo não fica parado em `processando`).